



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lab 6 — Cross-Course Capstone Project: SmartDoc AI

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI + Previous AI Courses
Session	Lab 6 — Cross-Course Capstone, 3 Hours + Take-Home Completion
Focus	Build one AI-powered Documents API that combines FastAPI Lectures 1–5 with prompt engineering, Gemini API integration, and a controlled tool-calling agent.
Evaluation	Capstone Project — 100%

Project Story

A small company wants a backend called SmartDoc AI. Users can create and upload documents, store them in SQLite, generate structured AI summaries with Gemini, and ask a read-only AI agent to retrieve information from the stored documents. The service must be documented, migrated, tested, and safe to run.

Today's Project Plan

- 1. Reuse the Lecture 4/5 Documents API foundation
- 2. Add secure Gemini integration and structured AI analysis
- 3. Build a small read-only tool-calling document agent
- 4. Test the full HTTP contract with Swagger, curl, and Postman
- 5. Optionally connect the FastAPI agent endpoint to n8n
- 6. Deliver a clean repository with migrations, tests, README, and review notes

01

What This Capstone Integrates

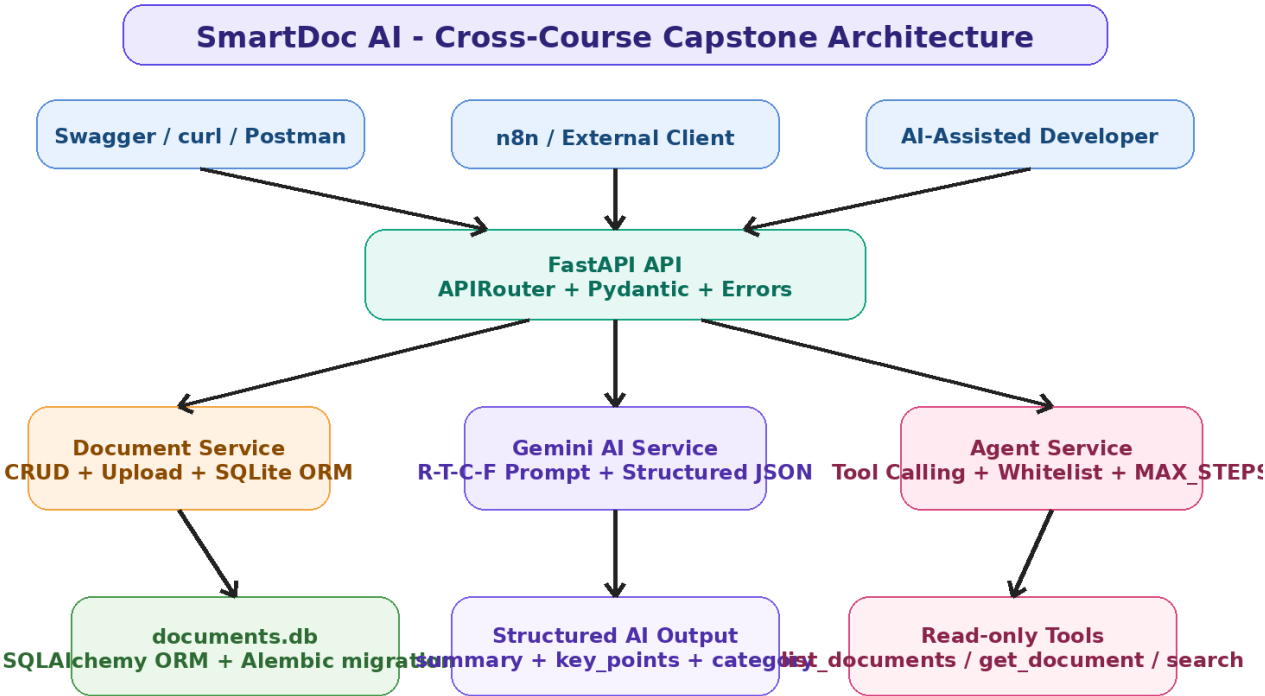
This is not a new isolated app. It is one project that deliberately reuses the concepts from the entire learning path.

Course / Lecture	Required Evidence in SmartDoc AI
GenAI & Prompt Engineering	R-T-C-F prompt structure, explicit output format, model limitations and safe handling of data.
AI-Assisted Development	Use an AI coding assistant for one scoped change, review the diff, correct mistakes, and document what you changed.
LLM API Integration	Load GEMINI_API_KEY from .env, call Gemini from Python, and request structured JSON output.
AI Agents Development	Expose narrow tools, validate arguments, whitelist tool names, use a MAX_STEPS guard, and let the model choose read-only actions.
FastAPI Lecture 1	GET/POST/PUT/DELETE, path parameters, JSON responses, status codes.
FastAPI Lecture 2	Pydantic validation, file upload, .env configuration, predictable errors.
FastAPI Lecture 3	SQLite + SQLAlchemy ORM + direct with Session(engine) as db CRUD.
FastAPI Lecture 4	Alembic migration, APIRouter organization, Swagger/ReDoc/OpenAPI.
FastAPI Lecture 5	HTTP contract testing with Swagger, Windows curl, and Postman.

02

Target Architecture & Required Files

Your final project should follow this high-level architecture. The AI features sit on top of the same FastAPI + ORM structure already built in Lecture 4.



Python Framework with FastAPI · Lab 6

Recommended project structure

```
smartdoc_ai/  
├── main.py  
├── database.py  
├── models.py  
├── schemas.py  
├── settings.py  
├── documents.db  
├── .env  
├── .env.example  
├── .gitignore  
├── requirements.txt  
├── README.md  
├── routes/  
│   ├── __init__.py  
│   ├── documents.py  
│   └── ai.py  
├── services/  
│   ├── gemini_service.py  
│   └── agent_service.py  
├── alembic.ini  
├── alembic/  
│   └── versions/  
└──
```

Start from the completed Lecture 4/5 project. Do not rebuild the API from zero.

Required HTTP endpoints

Method	Path	Purpose	Expected
GET	/health	Simple service health check	200
GET	/documents	List stored documents	200
POST	/documents	Create from JSON	201
GET	/documents/{id}	Read one document	200 / 404
PUT	/documents/{id}	Update one document	200 / 404
DELETE	/documents/{id}	Delete one document	200 / 404
POST	/documents/upload	Upload a text file and create a document	201 / 400

Keep the same direct database pattern used in Lectures 3–5:

```
with Session(engine) as db:
    doc = db.get(Document, document_id)
    ...
```

File-upload rule

- Required: accept text/plain. Read the uploaded text and store it in Document.content.
- Optional stretch: accept PDF only if you also implement a reliable text-extraction step. Do not pretend binary PDF bytes are readable text.

New migration requirement

Add one new nullable column named ai_summary to the Document ORM model. Generate and review an Alembic migration instead of deleting documents.db.

```
ai_summary: Mapped[str | None] = mapped_column(
    Text,
    nullable=True,
)

python -m alembic revision --autogenerate -m "add ai summary"
python -m alembic upgrade head
python -m alembic current
```

✓ CHECKPOINT — Stage A

What must still exist after the migration?

Answer: The old documents and the migrated description column, plus the new ai_summary column.

Create services/gemini_service.py. The API key must come from .env; never hardcode it in Python.

Python Framework with FastAPI · Lab 6

```
# .env.example
GEMINI_API_KEY=replace_me
GEMINI_MODEL=your_model_name
DATABASE_URL=sqlite:///documents.db
```

Add this endpoint:

```
POST /documents/{document_id}/analyze
```

It must load the document from SQLite, send its content to Gemini, and return structured JSON with this contract:

```
{
  "summary": "short summary",
  "key_points": ["point 1", "point 2"],
  "category": "technical | business | general",
  "suggested_priority": 1
}
```

Prompt requirement — use R-T-C-F thinking

Part	What the student should specify
Role	You are a careful document analyst.
Task	Summarize and classify the supplied document.
Context	Use only the provided document content. Do not invent missing facts.
Format	Return JSON with summary, key_points, category, suggested_priority.

- Save only the returned summary into Document.ai_summary and commit the change.
- If Gemini fails, return a controlled application error; do not expose secrets or raw internal tracebacks to the client.

Now connect the AI Agents course to the same backend. The agent is not allowed to delete or modify data. It can only inspect documents and answer from tool results.

Create these narrow tools

Tool	Input	Output
list_documents	none	id + title + priority for available documents
get_document	document_id > 0	the selected document fields
search_documents	query string	matching ids/titles from the local database

Create the endpoint:

```
POST /agent/ask

{
  "message": "Find the API notes and tell me their key idea"
}
```

Agent loop requirements

- Whitelisted tools only: the model may call only the three tools above.
- Validate tool arguments before execution.
- Use MAX_STEPS = 5 to prevent an accidental endless loop.
- Feed the tool observation back to Gemini, then continue until a final text answer is returned.
- Convert tool exceptions into safe observations; do not return database internals or secrets.
- Log tool name + success/failure for debugging.

```
MAX_STEPS = 5
TOOLS = {
    "list_documents": list_documents_tool,
    "get_document": get_document_tool,
    "search_documents": search_documents_tool,
}

for step in range(MAX_STEPS):
    # ask Gemini
    # if no tool call -> return final answer
    # validate + execute only if call.name is whitelisted
    # append observation and continue
    ...
```

✓ CHECKPOINT — Stage C

Who actually executes get_document when Gemini requests it?

Answer: Your Python application. The model proposes the tool call; your code validates and executes it.

Python Framework with FastAPI · Lab 6

Use Cursor, Copilot, or another coding assistant for exactly one scoped change. The purpose is not to outsource the project; it is to demonstrate the review workflow from the earlier AI-Assisted Development course.

Example prompt

```
In routes/ai.py, add POST /documents/{document_id}/analyze.  
Reuse the existing Document model and Session(engine) pattern.  
Return 404 when the document is missing.  
Do not change the existing CRUD routes.  
Keep the response compatible with AIAnalysisResponse.
```

Required evidence:

- Save the prompt you used.
- Review the generated diff before accepting it.
- Write 3–5 lines explaining one thing you verified or corrected manually (for example: wrong status code, hardcoded key, incorrect import, or architecture mismatch).

✓ CHECKPOINT — Stage D

What is the learning objective of using an AI coding assistant here?

Answer: Steer, review, and test AI-generated code rather than accepting it blindly.

07

Stage E — Test the Complete HTTP Contract

Use the same client-side testing discipline from Lecture 5. Do not call the route functions directly and stop there.

Test	Expected
GET /health	200
GET /documents	200
POST /documents valid	201
POST /documents invalid	422
GET /documents/999999	404
POST /documents/upload with unsupported type	400
POST /documents/{id}/analyze for existing id	200
POST /documents/{id}/analyze for missing id	404
POST /agent/ask with a document question	200 + grounded final answer

Windows CMD smoke-test examples

```
curl -i http://127.0.0.1:8000/documents

curl -i -X POST http://127.0.0.1:8000/documents ^
-H "Content-Type: application/json" ^
-d '{"title":"SmartDoc Notes","content":"FastAPI with AI","priority":2}'

curl -i -X POST http://127.0.0.1:8000/agent/ask ^
-H "Content-Type: application/json" ^
-d '{"message":"List the available documents"}'
```

Postman requirements

- Create a SmartDoc AI collection.
- Use base_url = http://127.0.0.1:8000 and a reusable document_id variable.
- Include success, 422, 404, upload-error, AI-analysis, and agent requests.
- Export the collection JSON for submission.

✓ CHECKPOINT — Stage E

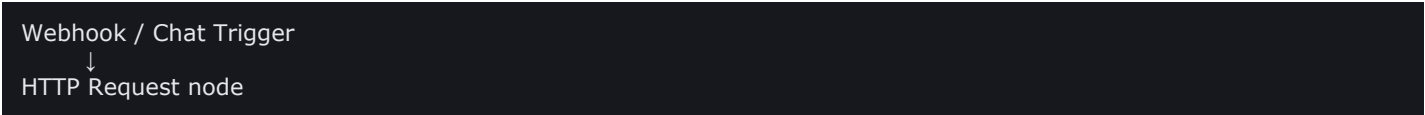
Is a 200 status alone enough to pass a test?

Answer: No. The actual JSON/body, documented response model, and expected behavior must also match.

08

Optional Hybrid Orchestration — n8n

For students who completed the visual orchestration course, connect n8n to the FastAPI service without moving the core logic out of Python.



Python Framework with FastAPI · Lab 6

POST http://127.0.0.1:8000/agent/ask

↓

FastAPI agent service

↓

Return response to n8n

- Keep FastAPI as the source of truth for database access and agent tools.
- Use n8n only as the event/integration layer.
- Do not connect destructive external tools unless you add an explicit approval step.

This section is a bonus extension; the core Lab 6 project is complete without n8n.

A submission is complete only when another developer can migrate, run, inspect, and test the project without guessing.

Area	Ready When...
Architecture	main.py assembles APIRouter modules; DB, schemas, AI service, and agent service are separated.
FastAPI	CRUD + upload + analyze + agent endpoints run and return JSON.
Validation	Pydantic handles request/response contracts and invalid data returns 422.
Persistence	SQLite data survives restart; direct Session(engine) pattern remains.
Migration	Alembic adds ai_summary without deleting the existing database.
AI	Gemini key is loaded from .env; structured document analysis is returned.
Agent	Only read-only whitelisted tools execute; MAX_STEPS guard exists.
Docs	/docs, /redoc, /openapi.json match the running API.
Testing	Swagger, curl, and Postman cover success and failure paths.
AI-assisted review	Prompt + reviewed diff note is included.
Security	No API key is committed; .env is ignored; error responses do not expose secrets.

Submission package

- Complete source code and folder structure.
- alembic.ini plus alembic/versions migration files.
- .env.example and .gitignore (never submit the real .env key).
- requirements.txt.
- Exported Postman collection JSON.
- README.md with: install → configure → migrate → run → test.
- Completed contract verification table with expected vs actual status codes.
- AI-assisted development review note (prompt + what you manually checked/corrected).
- Optional: exported n8n workflow JSON or screenshot.